Unsupervised Root-Cause Analysis in Multi-System Logs Using Embedding-Based Event Graphs*

Patricia Alonso del Real[1], Sam Afzal-Houshmand[2], Gaurav Choudhary[1], and Nicola Dragoni[1]

[1] DTU Compute, Technical University of Denmark, Lyngby, Denmark
[2] Copenhagen University, Denmark
patriciaalonso100@gmail.com, saho@adm.ku.dk {gauch, ndra}@dtu.dk

Abstract

Root-cause analysis (RCA) represents a challenge in security operations. This is because the evidence that explains the incident can be found in different (heterogeneous) systems (e.g., VPN/RADIUS, firewalls, host authentication), with different schemas, vocabularies, and timestamp precision. We investigate unsupervised RCA and frame it as a ranked log-evidence retrieval under realistic constraints: no labels, limited reliance on sensitive identifiers, and high background noise. We propose a pipeline that starts with an anchored alert and builds a temporal event graph where nodes are log events and edges encode plausible continuity. We compare a TF–IDF baseline against a self-supervised contrastive representation model trained on masked log templates with heuristic positive-pair mining. Also, we evaluate on a manually reconstructed "golden" SSH brute-force episode and report set-based and ranking-aware metrics, plus reachability diagnostics to separate connectivity from prioritization failures. Results show that simple continuity assumptions can make ground-truth evidence fully reachable, while precision is the main bottleneck due to noise. Contrastive learning has modest set-level gains, but improves early ranking of the core causal chain, suggesting learned representations help prioritization more than coverage in this setting.

1 Introduction

In the area of IT operations and cybersecurity, the reliability and security of services increasingly depend on complex distributed infrastructures. Multiple components can be involved in the cause of an incident (perimeter firewalls, identity and access management systems, endpoint agents, etc) [1]. Logs are a crucial source of evidence for analyzing faults and investigating the reasons that caused a security incident, but manual correlation can be slow and prone to error due to their heterogeneity and large volume [2].

1.1 What is Root Cause Analysis and How It Helps

Root-cause analysis (RCA) is the approach selected for this reconstruction of events in this work. In comparison with anomaly detection, where the goal is to 'flag' abnormal events, RCA's goal is to select and prioritize events that are most likely to explain how an incident occurred [3]. RCA narrows the search space to the events that are actually relevant, exposing relationships among them, which then helps remediation. For security incidents such as SSH brute-force

*Proceedings of The 2026 IFIP WG 8.4 International Symposium on E-Business Information Systems Evolution (EBISION 2026), Article No. 45, July 8–10, 2026, Seoul, South Korea.
© The copyright of this paper remains with the author(s).

compromises, RCA is especially valuable, and this is why it was chosen as the case study for this research: the causal chain frequently involves connection activity at network boundaries (e.g., firewall telemetry), authentication decisions (e.g., RADIUS logs), and the final host-level success event (e.g., an sshd login on a Linux system) [4]. Without systematic correlation, the investigator is left with an expensive process of searching, filtering, and mentally correlating evidence across systems.

One of the main challenges of multi-system RCA is the automation of the process. A large amount of proposed solutions is still rule-based or relies on annotated labels [5]. This approaches can be effective in narrow domains, but they still need a considerable amount of expertise on the domain and do not usually generalize across different vendors or environments. Labels are costly and usually unavailable in real-world scenarios as incidents are rare, diverse, and can have sensitive information. These limitations motivate a complementary line of work: unsupervised RCA, which does not use labeled data and does not rely on rules [6].

1.2 Problem Statement and Gaps in Research

One of the main obstacles in unsupervised RCA across different systems is semantic misalignment among logs from different sources [5]. Different systems can describe events that are related using different schemas and/or vocabularies. For instance, a firewall can log a connection that was allowed and the host can log a successful login. These events, even if they were caused by the same attacker and led to the same incident, the messages from the logs from both of the systems can appear unrelated at the surface level. There are sparse representations that can capture word overlap in an efficient way, like bag-of-words TF–IDF, but they still lack of cross-domain generalization [7].

This motivates the use of representation learning, particularly embedding methods that can map heterogeneous events into a shared vector space where semantically related events are closer together. However, applying representation learning in a fully unsupervised, multi-system RCA setting has its own difficulties: the training signal has to be computed without labels, the representations must avoid leaking sensitive identifiers, and improvements must be not only a better similarity structure but an improved root-cause retrieval, which translates into actual relevant logs appearing early in the explanation [8, 9].

1.3 Proposed Solution

We investigate embedding-based event graphs as the representation for heterogeneous logs in unsupervised RCA to address the challenges mentioned. The nodes in the graph are log events and the edges represent plausible relationships among these events based on shared actor and host contexts.

Within this context, we study whether contrastive representation learning could improve the prioritization of relevant events across systems in comparison to a sparse-text baseline. This baseline uses a text representation of log messages where they have been anonymized and templated, and then performs a simple anomaly scoring. Afterwards, it performs an alert-anchored traversal to retrieve a ranked set of candidate events. Our aim is to analyze how this baseline can be improved with contrastive learning over masked log templates in order to learn embeddings. The goal is not to compete directly with supervised state-of-the-art methods, but to answer a more operationally relevant research question: *When labels and handcrafted rules are unavailable, can learned representations improve unsupervised cross-system RCA beyond what can be done with simple lexical features?*. In this work, we use a specific security scenario:

SSH brute-force attack episodes collected across multiple streams on Copenhagen University's *Elasticsearch* platform.

We have three main contributions. We propose a data pre-processing pipeline that normalizes heterogeneous *Elasticsearch* logs into a canonical form, anonymizes sensitive identifiers (IPs, usernames, connection IDs), and produces structured and textual fields. Also, we develop a baseline that combines sparse-text features with temporal graph traversal, serving as a strong reference point to understanding what can be achieved without labels or handcrafted correlation rules. Finally, we perform an investigation on contrastive learning for semantic correspondence across different systems and analyze how different design choices led to different outcomes: positive-pair mining strategies, anomaly scoring choices, and temporal graph connectivity parameters.

1.4 Research Questions

This work has three main research questions:

- **RQ1:** How effectively do embedding-based event graphs represent multi-system security incidents for unsupervised RCA, and what connectivity assumptions are required to make root-cause evidence reachable from an alert anchor?
- **RQ2:** Does contrastive learning on masked log templates improve cross-system semantic alignment and evidence retrieval compared to a TF–IDF baseline?
- **RQ3:** Which design choices are the ones that determine the most the RCA performance and failure modes in real-world log data?

These questions make this work focus on performing an investigation on how event graphs need to be designed in order to have a plausible incident narrative, and under which assumptions the contrastive model outperforms the baseline, and why.

2 Related Work

Log-based RCA has become more relevant as modern infrastructures generate big amounts of telemetry data across different and heterogeneous systems. This heterogeneity makes manual correlation slow and, therefore, motivates automated RCA methods that can reconstruct the context that led to an incident [10, 11].

Prior work covers different settings and types of outputs. A considerable amount of AIOps RCA focuses on component or metric localization, producing a list of candidate services/metrics [12]. In contrast, log-based RCA focuses on retrieving explanatory log evidence. This work falls into the latter category: given an alert-centered episode, it performs a ranked evidence retrieval with an anchored alert over a temporal event graph built from logs from different systems.

This chapter is organized around three main axes: (i) the supervision regime (supervised, weakly supervised, unsupervised/self-supervised), (ii) the representation used for correlation (lexical features versus graph/temporal structures and learned embeddings), and (iii) the output unit (component/metric localization versus log-evidence retrieval).

2.1 Anomaly Detection vs. Root Cause Analysis

Anomaly detection and RCA are related, but perform different tasks in operational analytics [10]. Anomaly detection focuses on identifying events that are not usual, typically by learning

patterns of normal behaviour and then reporting anomalies, as in *Deeplog*, from Du et al. [13]. It has also been studied the use of transformers and self-supervised approaches to improve the quality of anomaly detection, for example, by learning contextual embeddings of log sequences in *Logbert*, from Guo et al. [14]. However, anomaly detection does not necessarily explain why an incident occurred. This is the core objective of RCA.

Previous works blend anomaly detection and RCA. For example, Wang et al. [15] use a pipeline with two stages: first, they detect anomalous events (or time windows), and then they rank candidates that led to the anomalies. This work also follows this two-stage pipeline, but with a different output unit (a ranked list of evidence items instead of a ranked list of metrics).

2.2 Log-Based RCA

Log-based RCA methods focus on retrieving log events that explain the cause of an incident. In this section we explain the two first axes around which this chapter is organized.

2.2.1 Supervision regime

The supervision regime used is one of the main dimensions of this chapter. Different supervision regimes have been used to achieve the explanation of an incident on a log-based design.

Unsupervised log-based RCA. There are several works that focus on producing explanations for incidents using logs without relying on labels. *Logrule*, from Notaro et al. [16], performs RCA on structured logs by investigating patterns that explain abnormal behaviour, but it does not build an explicit event graph to achieve this goal. Other approaches, like Lu et al.'s Spark work [17], use log features and custom heuristics to detect anomalies and identify likely causes, also lacking a temporal event graph.

These methods show that RCA can be performed without full supervision. However, they are either (i) domain-specific, or (ii) evaluated on settings where the inputs have a fixed structure instead of using multi-stream data where vocabularies and schemas differ due to vendor variability.

Weakly supervised and PU log-evidence ranking There are works where logs are the central part that use weak signals that are more available than event-level root-cause annotations. *LogLAB*, by Wittkopp et al. [18], even if it does not do RCA, envisions it by progressing from anomaly detection towards automated log labeling: from an incident window, it trains attention-based models so logs that are more likely to be the cause of abnormal behaviour are highlighted. Following this direction and applying RCA, *LogRCA* [10] uses Positive–Unlabeled learning, where the training data has some logs labeled as "positives" (relevant to incidents) and the remaining logs are not labeled. It uses this weak supervision to train models that rank log lines by how likely they are to be the cause (or one of the causes) of a fault.

These approaches are relevant because they have the same output unit as this work (ranked log evidence). However, they assume (i) a setting where logs can be correlated using stable, non-sensitive identifiers and platform metadata, and (ii) an environment that is semantically coherent in which log schemas and vocabularies are consistent, and cross-vendor or cross-layer semantic alignment is not one of the main challenges.

Regarding the first limitation, in this work's investigation the strongest correlators are often sensitive data (IPs, usernames, connection/session IDs). Therefore, they are masked or excluded to prevent shortcut learning and leakage. Regarding the second one, we address the problem

of semantic misalignment due to vendor-specific vocabulary. This motivates the self-supervised approach selected: the system learns cross-stream representations by correlating event templates using self-supervised positive pairs mined from anonymized actor/host continuity. Importantly, these continuity cues or other IDs are never part of the text embedding input, so the contrastive model encourages semantic alignment across streams instead of memorizing identifiers.

2.2.2 Representation used for correlation

Another dimension of this chapter is the representation used for cross-system correlation and ranking in log-based methods. What baselines like TF–IDF lack, while still useful and interpretable, is the capacity of capturing semantic similarity beyond overlap of tokens. Works like *LogBERT* [14] use transformer-based methods that learn embeddings that capture surrounding meaning of log sequences utilizing self-supervision. Surveys show that there is a tendency to move towards deep/self-supervised representations in order to assess the issue of evolving formats and to reduce feature engineering [19]. In heterogeneous telemetry, differences in vocabularies and/or schemas across different systems or vendors, which make them an unrealiable similarity flag, motivate the use of these representations.

For these reasons, this work uses self-supervised contrastive learning with *heuristic positive mining* and leakage control, which is especially relevant in security telemetry. The main objective is not to explicitly outperform benchmark systems, but to test if this model improves the prioritization of evidence that explains an incident in comparison to TF–IDF baselines.

2.3 Graph- and Representation-Based Correlation for RCA

Graphs are widely used in RCA because they explicitly represent dependencies and make it possible to perform a structured reasoning. This subsection shows representative graph-based approaches in AIOps where the output is a ranked list of services or metrics that are likely responsible for an incident. *CIRCA*, by Li et al. [12], performs microservice RCA using causal inference over service telemetry. *CausalRCA*, by Xin et al. [20], constructs causal graphs over metrics and ranks candidate root-cause metrics. *ART*, by Sun et al. [21], performs unsupervised RCA also ranking root-cause services/components. *CHASE*, by Zhao et al. [11], performs anomaly detection and RCA by building a causal hypergraph over multimodal microservice telemetry (logs, metrics, traces) and then ranking these service instances to localize the potential root-cause metrics.

These methods are important because they show strong design ideas (how to build the graph, how to reason causal links), but they have a different output unit from the one in this work. Component/metric localization assumes that the explanation for an incident is formed by one component/metric. Therefore, the mentioned in this section work normally uses AC@k or Avg@k for evaluation. In this work, the explanation is composed of multiple log events, so the most useful evaluation metrics are based on early precision, recall, and F1-score, although complementary metrics are also used.

Another difference is the information used to define the graphs' edges. In microservice settings, graphs often encode causal dependencies among services/metrics, like in *MicroRCA* [22]. In SOC telemetry, these dependency graphs are not usually available, so correlation has to be based on operational continuity cues. For this reason, the event graph in this work uses log events as nodes and has edges that encode actor- and host-centric temporal continuity.

This choice motivates an evaluation protocol that is not emphasized in most of the prior work: separating (i) connectivity failures, where relevant evidence is not reachable from the

alert under the assumed graph design choices, from (ii) prioritization failures, where evidence is reachable but is ranked with a too low suspicious score.

2.4 Evaluation Protocols

Works in the first table evaluate the ranked retrieval of logs, so they usually report Recall@k and Precision@k. Works in the latter typically assume that the explanation of an incident is a small root-cause set, so they report AC@k/Avg@k. We primarily follow the evaluation protocol of the first table.

Another issue to take into account is that the construction of the ground truth differs among settings. Benchmarks where faults are injected provide clean labels by design [20], while in real incidents there are ambiguous boundaries, duplicated log lines, and heterogeneous precission in timestamps. Therefore, we (i) manually reconstruct the ground truth using *Elasticsearch* and *Kibana*, (ii) make a distinction between *core* and *extended* evidence sets, (iii) report explanation sizes and analytics for both node-level and evidence-level (deduplicated) retrieval, and (iv) add diagnostics regarding reachability to separate limitations on connectivity and ranking quality.

3 Methodology

This chapter describes the methodological design of the thesis: how multi-system log data is collected and structured into episodes, how a manually reconstructed *golden episode* is created as ground truth, how the components are trained, and how the resulting RCA outputs are evaluated on a real scenario attack.

3.1 Overall Approach

The steps followed were:

1. **Selection of a real alert and definition of a golden episode:** We select one real *Potential Successful SSH Brute Force Attack* alert and extract logs from a fixed window of $[-27, +1]$ minutes around the alert timestamp. From here, we manually reconstruct the *golden episode*, which serves as ground truth and will be evaluated after running the pipeline.

2. **Collection of additional episodes for unsupervised training:** We collect multiple brute-force related alerts and extract the same fixed 28-minute window around each alert. To keep the episodes realistic, we extract both attack-related logs and background logs. Due to memory constraints, we sample streams with high volume while retaining as much as possible from the streams that form the incident "spine" (ISE/ASA/SSH host authentication).

3. **Training of unsupervised/self-supervised components:** We train (i) an unsupervised anomaly scoring model (baseline) and (ii) a self-supervised contrastive representation model. Both are trained only on non-golden episodes, without labels.

4. **RCA on the golden episode and evaluation:** We run the pipeline on the golden episode and evaluate how well the produced explanation recovers the manually reconstructed causal chain under strict (*core*) and tolerant (*extended*) ground truth definitions.

3.2 Manual Construction of the Ground Truth Causal Chain

To evaluate the outcome, we reconstruct the causal chain that led to a *Potential Successful SSH Brute Force Attack* alert that occurred on October 16th 2025 at 09:52:51.221. We use symbolic names for the IPs, user names and host names for privacy. The attacker internal IP is denoted as *IP1*, the attacker user name is denoted as *username*, the ssh host IP is denoted as *IP2*, and the host name is denoted as *hostname*. The reconstruction is based on (i) temporal ordering and (ii) entity continuity across systems.

The resulting ground truth narrative consists of four logical stages:

1. **ISE_VPN_ASSIGN.** Cisco ISE provides evidence that a VPN session is active and that the attacker was assigned an internal IP *IP1* during the incident window. In practice, RADIUS accounting *Interim-Update* events are beacons that inform about session-presence, used to confirm that the user with IP *IP1* had started a VPN session before performing the attack [23].

2. **ASA_FLOW_CREATION / ASA_FLOW_EXPIRATION.** Cisco ASA firewall logs record the lifecycle of TCP flows. In this setting, they prove that that connections from *IP1* to the SSH host (*IP2*, port 22) have been explicitly allowed. To validate this theory, we reconstruct TCP/22 flow intervals by correlating *flow-creation* and *flow-expiration* records. We then check that SSH authentication events from *IP1* happened within the period of time in which the firewall flow was active. Because ASA logs have millisecond precision while SSH logs are second-granularity, we apply a small tolerance around session boundaries [24].

3. **SSH_FAIL.** The SSH host authentication stream provides direct evidence of repeated *sshd* authentication failures with the actor IP being *IP1*. These events form the brute-force phase of the incident chain [25].

4. **SSH_SUCCESS.** After multiple failed authentication attempts, there is a single successful *sshd* authentication from IP1. This success event, along with the previous fails, is the cause of the alert and serves as the most direct host-level indicator of the incident in the episode [25].

Core vs. extended ground truth. As mentioned, we differentiate between two types of *ground truth*: *core* vs. *extended*. The first one the minimal and completely certain causal path that explains the incident, and the latter includes plausible evidence that could belong to the causal chain, but not with 100% certainty.

3.3 Data Extraction

We retrieve the logs for each of the episodes using *Elasticsearch* using Python scripts. We query a fixed time window of 28 minutes (27 minutes before the alert and 1 minute after, when applies) and export the per-stream logs into JSON files.

For feasibility reasons, we apply filters to certain streams to retain the most incident-relevant logs, while keeping the extraction procedure consistent across episodes. Also, several streams generate millions of events within the stated time window. For this reason, we set a fixed budget for each episode of approximately 30,000 events. This makes it easier to manage the data and to compare across episodes.

Streams relevant to the incident (Cisco ASA, Cisco ISE, and authentication system) are prioritized, so we try to keep as much as possible from these streams. After applying stream-specific filters to focus on SSH/VPN/flow evidence, we retain all core events when feasible. If the volume of data is still too high, then we set controlled caps and sample within the stream. The remaining budget is used to include activity from other streams that contribute with background noise at host and network level. To prevent a single noisy stream from dominating the episode, we use equal per-stream quotas for the non-core streams.

3.3.1 Stream Selection per Episode

To evaluate the RCA model under realistic multi-system noise, each episode includes seven *Elasticsearch* data streams:

- **Core streams** $\mathcal{S}_{\text{core}}$: Cisco ASA, Cisco ISE, and the authentication system.
- **Context streams** $\mathcal{S}_{\text{context}}$: Palo Alto firewall, Filebeat shipper, operating-system security logs, and endpoint network telemetry.

3.3.2 Episode Budgeting and Sampling Strategy

Some streams produce millions of logs in just 28 minutes. This makes the process of feature extraction, embedding, and construction of the graph not feasible with the available resources. For this reason, we enforce a budget for each episode of approximately $B = 30{,}000$ logs.

This choice is a data management choice, not a labeling shortcut. We do not use ground truth annotations to sample the data, and we do it based on each stream identity and internal structure.

Core streams. We restrict the *Cisco ASA* stream to SSH-relevant firewall flow evidence (TCP destination port 22/2222) with the requirement of a connection identifier used to build flow intervals. We sample *Cisco ISE* stream using session-aware sampling to avoid a single long VPN session dominating the episode. Regarding the authentication system, we prioritize *sshd* authentication events and keep a bounded amount of events with different processes (e.g., *sudo*, *systemd*) to capture plausible post-authentication activity.

Context/non-core streams. Within each context stream, we use time-stratified sampling across the episode window (minute-level strata) to preserve temporal coverage and avoid concentrating the sample in periods where there is a very high amount of activity.

Episode-level structuring. Raw logs are exported from *Elasticsearch* as JSON files, one for each type of stream, for each 28-minute episode window. Episodes are treated as the unit of training and evaluation, where RCA is performed.

Canonicalization across streams. Events from different streams have different schemas and vocabularies. In order to make them comparable, we convert each raw log into a canonical representation or template that standardizes fields that are common to every stream, and then keeps the ones specific to each one. This representation is used throughout the pipeline to ensure controlled comparisons between baseline and contrastive variants.

Identifier anonymization to prevent leakage. To avoid trivial shortcuts and to make the model less affected by identifiers specific to certain environments, tokens that can potentially identify an actor or process (e.g., raw IP addresses, usernames, firewall connection IDs) are excluded from the templates used as inputs. These identifiers are anonymized and converted into pseudonyms. This enforces that the models have to rely just on behavioral and semantic patterns instead of memorizing specific identifiers.

Duplicate handling at the evidence level. Sometimes, the exports include exact duplicate records. To avoid artificially inflating the size of our explanations and the evaluation counts, we group duplicates together into evidence items.

Outcome. The outcome of this stage is mainly a unified *parquet* file with all of the per-episode structured datasets. The steps to achieve this oitcome are deterministic given a fixed configuration, so experiments can be reproduced.

3.4 RCA Pipelines

We evaluate two RCA pipelines that share the same high-level retrieval procedure but have different event representation used for scoring.

3.4.1 Common RCA Retrieval Procedure

Both pipelines use the same setup for handling incident responses: Starting with an alert and the time window around it, the system has to construct a temporal multi-system event graph, and then produce an explanation as a prioritized list of key events.

- **Inputs:** An episode composed of log events from different streams and an alert timestamp.
- **Graph:** Representation where the nodes are events, and the edges encode temporally plausible continuity relationships.
- **Alert:** The alert is the starting point of the explanation.
- **Retrieval objective:** Returns a ranked list of events that best reconstruct the chain of events that caused the alert to be triggered.

3.4.2 Baseline Pipeline: Lexical Representation + Unsupervised Scoring

The baseline pipeline served as a strong unsupervised reference. Every event was represented using the standardized templates. An unsupervised anomaly detector was trained on non-golden episodes and then tested on the golden episode to assign each event a suspiciousness score. The RCA retrieval procedure then used these scores as the ranking signal for the explanation retrieval.

This baseline answered the question: *How well can an unsupervised lexical model prioritize events relevant to an incident for RCA without any representation learning?*

3.4.3 Contrastive Pipeline: Self-Supervised Semantic Alignment + Unsupervised Scoring

The contrastive pipeline replaces the baseline representation with embeddings trained to improve semantic alignment across heterogeneous streams. Events are also represented using the normalized templates, but from here we first obtain dense embeddings (via dimensionality reduction), and then train a lightweight mapping so that events that are likely part of the same incident context are placed close together in the embedding space, while events that are likely not related are pushed apart.

We train using self-supervised learning by mining *heuristic positive pairs* within episodes. This encourages related events to be closer in the embedding space, while pushing the rest apart. After training, the learned embeddings are used as input to the same type of unsupervised anomaly scoring as in the baseline, producing suspiciousness scores for every event.

This pipeline answers the question: *Does learning a cross-stream semantic space improve the retrieval of a causal chain compared to lexical baselines, under a controlled RCA procedure?*

4 Results

Here we portray the main results of the experiments.

4.1 Experimental Set Up

We evaluate the proposed RCA pipeline as a *ranked retrieval* problem over log-event nodes. Given the episode e, the method outputs an ordered list of nodes (an "explanation") produced by a backward walk from the alert node in the constructed temporal graph.

Notation. Let $R_e = [r_1, r_2, \ldots, r_m]$ be the ranked list of nodes returned by the RCA algorithm for episode e, where r_1 is the highest-ranked node. Let $\hat{R}_e = \{r_1, \ldots, r_m\}$ be the corresponding set of returned nodes (ignoring order).

Different streams have different timestamp precisions (e.g., Cisco ASA flow events with millisecond timestamps versus SSH logs without millisecond information). For this reason, we have two definitions for *ground truth*:

- **Core ground truth** G_e^{core}: the minimal set of logs that are considered essential to explain the incident.
- **Extended ground truth** G_e^{ext}: a set with a larger size that includes the core logs plus additional logs that could reasonably be part of the same causal narrative.

The alert node is excluded from evaluation. It is removed from R_e and $\hat{R}_e$ before computing all metrics.

Set-based retrieval metrics. Let e be an episode, $\hat{R}_e$ the set of retrieved log-event nodes for e, and let G_e denote the ground truth evidence set for e (either G_e^{core} or G_e^{ext} depending on the protocol). We compute precision, recall, and F1-score on the returned set:

$$P_e = \frac{|\hat{R}_e \cap G_e|}{|\hat{R}_e|}, \qquad R_e = \frac{|\hat{R}_e \cap G_e|}{|G_e|}, \qquad F1_e = \frac{2P_eR_e}{P_e + R_e}.$$

We also report the size of the retrieved explanation. Recall can be inflated when many nodes are retrieved, so by reporting this value we can see the explicit trade-off.

Because the RCA algorithm operates on log-event nodes in the graph, we first define the node-level explanation size as:

$$S_e^{\text{nodes}} = |\hat{R}_e|.$$

In addition, because some logs are duplicated in the exported data (same timestamp, data stream, source IP, destination IP, host name, event action, event outcome, event code and destination port), we also report an evidence-level explanation size after collapsing duplicates into a single evidence item:

$$S_e^{\text{evidence}} = |\hat{R}_e^{\text{evidence}}|.$$

Ranking-aware early retrieval metrics. In addition to retrieving the correct nodes, the ranking should output relevant evidence early. For a budget k, let $R_e@k = [r_1, \ldots, r_k]$ be the first k nodes of the ranked output and $\hat{R}_e@k$ its set version. We compute Precision@k, Recall@k and Hit@k:

$$P_e@k = \frac{|\hat{R}_e@k \cap G_e|}{k}, \qquad R_e@k = \frac{|\hat{R}_e@k \cap G_e|}{|G_e|}.$$

We report these values for $k \in \{5, 10, 20, 50\}$, values chosen to reflect realistic inspection budgets.

It is important to mention that several RCA papers report an *accuracy@k* metric (AC@k) for ranked root-cause localization. This metric is equivalent to precision@k when $k \leq |V_{rc}|$ and to recall@k when $k > |V_{rc}|$, so we directly use these two latter metrics.

4.2 Main Results

All experiments in this chapter are done using the manually annotated golden episode. We evaluate two pipelines:

- **Baseline:** TF–IDF features over masked templates, Isolation Forest anomaly scoring, temporal graph edges (actor/host), and backward RCA walk with priority $(-score_norm, |\Delta t|)$.
- **Contrastive model:** Contrastive representation learning over TF–IDF+SVD base embeddings (projection head trained with an NT-Xent style objective on mined positive pairs), followed by anomaly scoring in the learned space and the same graph construction and walk strategy.

Unless stated otherwise, results are reported using the evidence-collapsed protocol and the alert node is excluded.

4.2.1 Set-based Retrieval and Explanation Size

Table 1 reports the set-based precision/recall/F1 and explanation sizes for core and extended ground truth (results in bold when the contrastive model outperforms the baseline or does not underperform). Both methods retrieve the same number of true positives (TP) for core and extended ground truth, hence identical recall. The contrastive method returns fewer evidence items overall, yielding higher precision and F1.

Method	S^{nodes}	S^{evidence}	Core TP	Core P	Core R	Core F1	Ext TP	Ext P	Ext R	Ext F1
Baseline	199	174	24	0.138	**0.750**	0.233	32	0.184	**0.800**	0.299
Contrastive	199	168	24	**0.143**	**0.750**	**0.240**	32	**0.190**	**0.800**	**0.308**

Table 1: Set-based retrieval metrics and explanation size.

4.2.2 Ranking-aware Early Retrieval

Table 2 reports ranking-aware early retrieval metrics (results in bold when the contrastive model outperforms the baseline or does not perfom worse). The contrastive model improves early retrieval for the *core* ground truth (notably at $k = 5$ and $k = 10$), but it degrades early retrieval for the *extended* ground truth. At $k = 50$, both methods have the same values for recall for both the *core* and *extended* ground truth. This indicates that the main contribution of contrastive learning in this setting is the prioritization of evidence instead of a broader coverage.

In terms of Hit@k, both methods achieve Hit@$k = 1$ for $k \in \{5, 10, 20, 50\}$ under both core and extended ground truth, which means that at least one relevant evidence item is present among the top-k results; therefore, the differences between methods are better reflected by $P@k$ and $R@k$ (how many relevant items appear early) rather than first-hit success.

Method	**Core** (G_e^{core})				**Extended** (G_e^{ext})			
	$P@5$	$R@5$	$P@10$	$R@10$	$P@5$	$R@5$	$P@10$	$R@10$
Baseline	0.200	0.031	0.400	0.125	0.800	0.100	0.900	0.225
Contrastive	**0.400**	**0.063**	**0.700**	**0.219**	0.400	0.050	0.700	0.175
	$P@20$	$R@20$	$P@50$	$R@50$	$P@20$	$R@20$	$P@50$	$R@50$
Baseline	0.550	0.344	**0.420**	**0.656**	0.850	0.425	**0.540**	**0.675**
Contrastive	0.450	0.281	**0.420**	**0.656**	0.750	0.375	**0.540**	**0.675**

Table 2: Ranking-aware metrics.

Across both methods, set-based recall is high for a fully unsupervised pipeline (0.75 core, 0.80 extended) but precision is low, reflecting that the backward walk returns a relatively large explanation set. The contrastive model shows a small but consistent gain in set-based precision/F1 by reducing the number of returned evidence items. Its most visible benefit is in early retrieval for the core chain (e.g., $P@10$ rises from 0.40 to 0.70), while it neglects some extended-context evidence in the top ranks.

Due to different datasets used and output units produced, this model can only be quantitatively compared to the baseline and not to other previous works. Therefore, it has been discussed in Section 2 in a qualitative way.

4.2.3 AUPRC over reachable evidence.

We report an additional diagnostic for ranking quality: the area under the precision–recall curve (AUPRC). We compute this over the *reachable* evidence pool (6,014 items). We use the suspiciousness score as the ranking signal. Due to class imbalance ($|G^{\text{core}}| = 32$ and $|G^{\text{ext}}| = 40$ are drastically smaller than the evidence pool), absolute AUPRC values are low, but both

methods are above the random baseline. We can see in Figure 1 that the contrastive model improves in comparison to the baseline: AP_{core} increases from 0.0324 to 0.0551, and AP_{ext} from 0.0341 to 0.0533. This is consistent with the ranking-aware early retrieval metrics improvements.

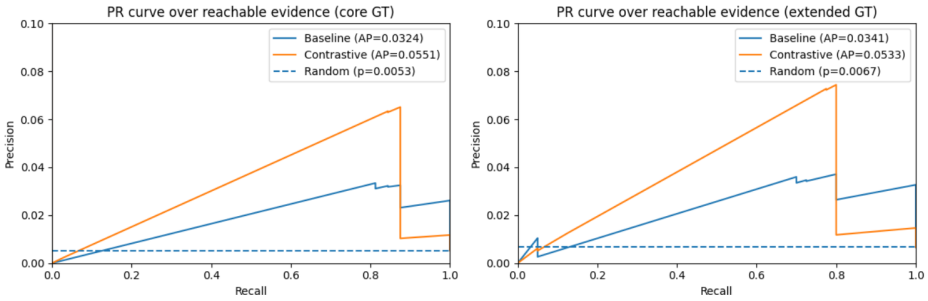


Figure 1: AUPRC for baseline and contrastive pipelines

.

4.3 Experiments

This section consists of several experiments where we address the research questions proposed.

4.3.1 Experiment A: Evidence Reachability

A necessary condition for RCA to be successful is that relevant evidence is reachable in the constructed graph. If a ground truth (GT) evidence item cannot be reached under the mechanics used by the RCA walk, then it does not matter which scoring or ranking strategy one uses because it will not be able to retrieve it. We therefore evaluate *reachability* as an upper bound on retrieval performance based on only graph connectivity assumptions.

Reachability definition. We compute the set of reachable nodes by running the same traversal mechanics as the RCA walk, but removing all ranking and budget effects. We initialize the traversal from the successors of the alert node and expand backwards by repeatedly visiting predecessors of visited nodes. In this diagnostic, we ignore anomaly scores and the maximum-node budget, but we preserve the temporal feasibility constraints (maximum lookback and forward slack) and the hop limit. The resulting reachable set represents all evidence that could be returned by the RCA walk in principle.

To be consistent with the main evaluation, we report reachability at the evidence level by collapsing nodes into evidence items. We define *reachable recall* for a ground truth set G_e as:

$$\text{ReachRec}(G_e) = \frac{\left|\mathcal{R}_e^{\text{evidence}} \cap G_e^{\text{evidence}}\right|}{\left|G_e^{\text{evidence}}\right|},$$

where $\mathcal{R}_e^{\text{evidence}}$ is the set of reachable evidence items.

Experimental factors. We ablate (i) edge types (actor edges vs. host edges) and (ii) temporal gaps for actor and host edge construction. All experiments are run on the golden episode using the same alert attachment strategy and the same time-window constraints as in the main RCA pipeline.

Results. Table 3 summarizes the reachability diagnostics. With both edge types enabled under the default temporal gaps (actor gap 120 s, host gap 60 s), the graph achieves perfect reachability for both the core and extended GT sets (32/32 and 40/40 reachable). An important insight is that actor edges are sufficient to obtain the same perfect reachability (actor-only also yields 32/32 and 40/40), while host edges alone are insufficient (host-only reaches only 21/32 core and 21/40 extended evidence). Without both actor and host edges, reachability collapses (1/32 core and 1/40 extended), confirming that explicit temporal connectivity assumptions are required to make evidence reachable from the alert anchor.

The actor-gap modifications show a clear threshold effect (table omitted for space): restricting the actor temporal gap to 30–60 s reduces reachable recall to 0.75 for both core and extended GT, while gaps ≥120 s restore perfect reachability. In contrast, the host-gap modifications maintain perfect reachable recall across tested values, but strongly change the size of the reachable evidence set. For example, host_gap=30 s leads to a perfect reachability while substantially reducing the reachable evidence pool compared to host_gap=60 s, indicating that host edges primarily control context expansion rather than being necessary for connecting the core incident chain in this episode.

Overall, this experiment answers RQ1 by showing that full GT reachability is achievable under simple connectivity assumptions, but it is dependent on including actor-based temporal edges and also using sufficiently permissive temporal gaps to bridge cross-system delays. It also answers RQ3 by showing that the same assumptions that guarantee reachability can considerably expand the candidate evidence pool, which leads to a lower precision.

Variant	**ReachRec$_{\text{core}}$**	**ReachRec$_{\text{ext}}$**	**Reachable evidence**
Both edges	1.000 (32/32)	1.000 (40/40)	6014
Actor-only	1.000 (32/32)	1.000 (40/40)	227
Host-only	0.656 (21/32)	0.525 (21/40)	28
No edges	0.031 (1/32)	0.025 (1/40)	1

Table 3: Reachability diagnostics (applies to both baseline and contrastive model).

4.3.2 Experiment B: RCA Walk Budget and Hop Limit

There are two hyperparameters that modify the trade-off between the size and the quality of the explanation: the node budget (*rca_max_nodes*), which determines maximum amount of nodes that can be selected, and the hop limit (*rca_max_hops*), which determines how deep the walk can expand through predecessor links. We perform two experiments according to these two hyperparameters and report results for both the TF–IDF baseline and the contrastive model.

Hop sensitivity. To see the effect of different hop caps, we fix a large node budget (*rca_max_nodes* = 5000) so the maximum number of nodes is not a constraint here. We experiment with *rca_max_hops* ∈ $\{1, 2, 3, 5, 10, 25, 50, 100, 150, \textit{None}\}$ (table omitted for space).

When the hop limit is very small, we recover almost none of the ground truth. As the hop limit increases, size of the explanation becomes more usable. When it has a value around *rca_max_hops* = 10–25, the explanation size is comparable to the ones observed when moderate node caps are chosen, as seen in Table 4. For instance, when the hop budget is 25, the baseline retrieves 60 nodes ($S^{\text{evidence}} = 46$) and the contrastive model retrieves 78 nodes ($S^{\text{evidence}} = 59$).

These are numbers on the same order as the $N = 50$–100 node-capped regime ($S^{\text{evidence}} \approx 37$–$82$).

With larger hop limits ($(\geq 100$) or no limit at all, the walk selects thousands of nodes. This is not feasible in real-world scenarios, but shows that the graph can perform a very deep traversal (the baseline has max hop selected = 527 and the contrastive model 386).

To characterize how "deep" the walk actually goes under each hop cap, we report hop diagnostics for the selected nodes: the maximum hop depth and the 90th percentile hop depth ($P90$), which summarizes the typical upper-range traversal depth while being less sensitive than the mean to a small number of very deep selections. At *rca_max_hops* = 25, $P90$ is already close to the cap (baseline $P90 = 24.0$, contrastive $P90 = 22.3$), indicating that most selected evidence requires multi-hop traversal rather than only immediate predecessors of the alert.

Node budget sensitivity. Now we fix the maximum number of hops to *unlimited* and experiment with *rca_max_nodes* $\in \{25, 50, 100, 150, 200\}$. In this regime, we increase coverage by increasing the node cap, but this eventually degrades precision due to background noise, as seen in Table 4.

With small node limits, the explanations are incomplete. For example, at $N = 25$ the contrastive run returns $S^{\text{evidence}} = 18$ and the baseline returns $S^{\text{evidence}} = 17$. This lowers recall and, consequently, F1-score, even if early precision has a high value (baseline has $P@10 = 0.900$ and the contrastive model has $P@10 = 0.700$, both in the *extended* set). At $N = 50$, the walk retrieves most of the relevant evidence while filtering noise. With this node limit, the contrastive model has a better F1-score in both ground truth definitions as it returns a smaller explanation set. This suggests that the contrastive model reduces the amount of items that are not relevant to the incident in the retrieval without sacrificing coverage.

We can see that if we increase N, we get larger explanations (contrastive: $S^{\text{evidence}} = 168$, baseline: 174), which give us a lower set-level precision and, consequently, F1-score. We can also see that the hops increase with the node budget (e.g., contrastive max hop selected grows from 10 at $N = 25$ to 119 at $N = 200$; baseline grows from 16 to 126). This makes sense: the more nodes are allowed, the further into the graph the walk will expand. In addition, this is reported in the $P90$ *hop selected* results, which shows typical upper-range depth of the graph traversal: for the baseline it increases from $P90 = 14.0$ at $N = 25$ to 116.2 at $N = 200$, and for the contrastive model from 9.0 to 110.2 (Table 4).

Model	N	S^{evidence}	$\mathbf{F1}_{\text{core}}$	$\mathbf{F1}_{\text{ext}}$	$P@10_{\text{core}}$	$P@10_{\text{ext}}$	**P90 hop sel.**	**Max hop sel.**
Node budget sensitivity (fixed rca_max_hops = None)								
Baseline	25	17	0.367	0.526	0.400	0.900	14.000	16
Contrastive	25	18	0.360	0.517	0.700	0.700	9.000	10
Baseline	50	41	0.575	0.667	0.400	0.900	30.200	35
Contrastive	50	37	0.609	0.701	0.700	0.700	16.000	17
Baseline	200	174	0.233	0.299	0.400	0.900	116.200	126
Contrastive	200	168	0.240	0.308	0.700	0.700	110.200	119

Table 4: Node budget sensitivity of the backward RCA walk under unlimited hops.

In summary, we can see that hop and node limits affect the returned explanations differently. Small hop caps prevent the walk from retrieving incident evidence even when it is reachable. Once the hop limit is high enough (roughly $\geq$ 10–25 in this setting), the node budget becomes

the dominant hyperparameter: if it is too small, it leads to under-explanation; if it is too large, it will introduce too much noise.

The results seen in Section 4 are done with a fixed node budget of $N = 200$ to ensure that the evaluation across pipelines is consistent. We can see in this experiment that a budget of $N = 50$ produces explanations with sizes that make the analysis more feasible and improves the trade-off between budget and quality for both methods. We then interpret the node limit from the main evaluation as a reference point in terms of explanation size.

5 Discussion

In this thesis, SOC root-cause analysis was treated as a task of retrieving ranked evidence that could best explain the cause of an alert using logs from multiple systems. The results on the golden episode show that, by using an unsupervised setting, meaningful narratives can be obtained. The main constriction was not finding the evidence, but whether that evidence could be retrieved early and with an acceptable amount of noise.

5.1 Precision as Bottleneck

Both pipelines (TF–IDF and contrastive) achieved a high recall under the set-based retrieval protocol, but a low precision because of the large size of the explanation set returned, as seen in Table 1. This situation corresponds to a realistic SOC failure mode: the system can retrieve a lot of evidence relevant to the incident, but it also returns a large amount of background context, which leads to an expensive inspection of the explanation.

The contrastive pipeline did not dramatically improve the baseline in set-based precision and F1-score, but it still enhanced it by performing a desirable mechanism: it reduced the explanation size without lowering recall. This means that, with this configuration, this self-supervised model helps filter some noise instead of reaching new evidence that caused the incident.

5.2 Main Value of Contrastive Model

The main improvement of the contrastive pipeline can be seen in ranking-aware metrics for the core causal chain, which Table 2 shows, especially at small inspection budgets ($k = 5$ and $k = 10$). This is more important than small set-level improvements because, in practice, analysts' workflows are budgeted [26].

At the same time, ranking-aware metrics for the extended causal chain degrade. This means that the contrastive signal makes the ranking of evidence prone to select the minimal causal explanation instead of broader contextual evidence. This shows a trade-off between between two explanation goals that are both valid: (i) the minimal core explanation, and (ii) the supporting evidence that is also plausible as part of the incident narrative.

5.3 Reachability vs. Prioritization

An important point in the methodology of this work is the distinction between connectivity and ranking. Table 3 shows that all of the ground truth evidence items are reachable from the alert anchor under the default graph design. This is relevant because it shows that the graph design is not the limiting factor for recovering explanatory evidence items.

Nevertheless, this experiment also shows that this high reachability can highly expand the pool of candidates, so precision degrades unless either the scoring function is strong or the expansion of the graph is controlled.

We can get an actionable insight from the edge ablations: actor edges alone are enough to have a perfect reachability, while host edges alone are insufficient. Operationally, a reasonable approach would therefore be to use host edges to expand context when the core explanation is incomplete.

This analysis shows that a calibration across heterogeneous sources is necessary for this task. In particular, some approaches that could reduce noise are:

- **Calibrate scores per stream:** We normalized scores globally, but different streams can have very different score distributions.
- **Control how the graph expands:** We could modify the walk so it starts with only actor edges and small time windows, and then expand to broader context if the explanation is incomplete.

Section 4 pointed at this: high suspiciousness scores in noisy streams led to false positives, while some core items that were not retrieved as part of the causal chain had comparatively lower scores.

5.4 Walk Budget and Hop Limit

Subsection 4.3.2 from Section 4 shows that limitations in hops and nodes to be reached influence differently the failure modes of the graph traversal. With a node budget high enough for it to be non-binding, small hop caps miss most of the relevant evidence to the causal path. Once the hop cap is large enough, the explanation size becomes to a the analyst-feasible regime that can be seen where moderate node caps are used. Too large hop limits (or no limit) lead to an explanation with a lot of background noise.

When hops are not restrictive, the node budget becomes the main operational constraint. Increasing this number improves coverage at the beginning, but beyond a budget $N = 50$, precision and F1-score degrade due to noise in the explanation set. In our main evaluation, N is fixed to 200. This experiment shows that smaller node budgets (around $N = 50$ in this episode) have a better trade-off regarding budget and quality. For this reason, the main setting is seen as a reference point for comparing pipelines, while we acknowledge the superior operational performance of this tune in this hyperparameter (node budget).

5.5 Limitations and Future Work

One of the main limitations of this work is that the quantitative evaluation is done on a single manually annotated golden episode. This can be enough to validate the mechanics of the pipeline, leakage control, and the distinction between reachability and ranking, but it still limits how far we can generalize the results to other types of incidents and environments. More incident types where not possible to analyze due to time constraints.

Another limitation is that episode construction and sampling were necessary due to the high volume of data. This is realistic in SOC telemetry, but it can affect which context evidence is available for it to be retrieved and ranked.

In addition, there is ambiguity in the ground truth protocol (core vs. extended) because of timestamp precision mismatches and duplicates. The fact that early retrieval behaves differently

for the core vs. extended ground truth is best seen as a consequence of how we defined and evaluated "core" and "context", not just as the model behaving inconsistently.

In summary, we obtain three conclusions from the reported results: (i) continuity-based event graphs can make SOC telemetry completely reachable from the alert in, at least, one realistic incident, (ii) the most dominant challenge is to control the noise from heterogeneous streams with high volume, and (iii) contrastive learning mainly helps improve early prioritization of the core causal chain rather than increasing the total coverage. Future work should evaluate more types of incidents and explore per-stream calibration. Also, it should investigate on a graph expansion where the core evidence is retrieved first, and the context evidence later in order to have a smaller explanation size and improve precision without lowering reachability.

6 Conclusion

In this work we investigated whether a self-supervised RCA pipeline could reconstruct the causal path that led to a security incident using contrastive learning from heterogeneous logs and without using labels, and whether this model's performance was better than a TF–IDF baseline (and why or why not). The approach proposed is an alert-anchored retrieval of ranked evidence over a temporal event graph where the edges encode operational continuity of actors and hosts.

The results on the manually reconstructed golden episode show that this pipeline achieved a high recall, but precision remained low because relatively large explanations were retrieved by the graph walk. The contrastive representation did not improve coverage of causal evidence, but it improved the prioritization of the core causal chain. However, the early retrieval of the extended causal chain worsened, highlighting that the understanding of what a "good root-cause analysis" depends on if the objective is to obtain a concise causal chain or a richer contextual narrative.

An important insight is the distinction we made between connectivity and reachability. Reachability diagnostics show that all of the evidence items can be reached under simple connectivity assumptions and, furthermore, host edges are not needed to achieve this task. Using both actor and host edges make the candidate pool considerably larger, which harms precision. Therefore, the main reasoning of this work is supported: in multi-stream SOC telemetry, the main bottleneck is not just representation learning, but reducing background noise while keeping the causal chain that led to the incident.

In summary, the main contributions of this work are:

- A method for root-cause analysis in SOC using event graphs that are anchored in alerts.
- A self-supervised learning pipeline with leakage control for semantic alignment across different systems.
- An evaluation protocol that distinguishes reachability from prioritization and core from extended evidence.

For future work, the evaluation should be extended beyond just one attack. In addition, one should explore stream-aware calibration and new graph design choices in order to reduce false positives without sacrificing reachability.

References

[1] Zahra Amiri, Arash Heidari, Nima Jafari Navimipour, and Mehmet Unal. Resilient and dependability management in distributed environments: A systematic and comprehensive literature review. *Cluster Computing*, 26(2):1565–1600, 2023.

[2] Jan Svacina, Jackson Raffety, Connor Woodahl, Brooklynn Stone, Tomas Cerny, Miroslav Bures, Dongwan Shin, Karel Frajtak, and Pavel Tisnovsky. On vulnerability and security log analysis: A systematic literature review on recent trends. In *Proceedings of the International Conference on Research in Adaptive and Convergent Systems*, pages 175–180, 2020.

[3] Machine Learning. Adaptive anomaly detection and root cause analysis by fusing semantics. *The Semantic Web: ESWC 2018 Satellite Events: ESWC 2018 Satellite Events, Heraklion, Crete, Greece, June 3-7, 2018, Revised Selected Papers*, 11155:272, 2018.

[4] Sachin Kumar Singh, Shreeman Gautam, Cameron Cartier, Sameer Patil, and Robert Ricci. Where the wild things are:{Brute-Force}{SSH} attacks in the wild and how to stop them. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*, pages 1731–1750, 2024.

[5] Feng Fu, Hong Ding, Yong Qin, Jian Yu, and Dechao Xu. Leveraging multi-agent framework for root cause analysis. *Complex & Intelligent Systems*, 12(1):4, 2026.

[6] Renjian Pan, Xin Li, and Krishnendu Chakrabarty. Unsupervised root-cause analysis with transfer learning for integrated systems. In *IEEE VLSI Test Symposium*, 2021.

[7] Jantima Polpinij, Thananchai Khamket, Chumsak Sibunruang, Anirut Chottanom, Jatuphum Juanchaiyaphum, Theeraya Uttha, Satitiphong Ua-Areemit, Bancha Luaphol, and Khanista Namee. Beyond domain dependency: Evaluating machine learning models for cross-domain sentiment classification. In *International Conference on Information and Communication Technology for Intelligent Systems*, pages 265–275. Springer, 2025.

[8] Egil Karlsen, Xiao Luo, Nur Zincir-Heywood, and Malcolm Heywood. Large language models and unsupervised feature learning: implications for log analysis. *Annals of Telecommunications*, 79(11):711–729, 2024.

[9] Roozbeh Aghili, Heng Li, and Foutse Khomh. Protecting privacy in software logs: What should be anonymized? *Proceedings of the ACM on Software Engineering*, 2(FSE):1317–1338, 2025.

[10] Thorsten Wittkopp, Philipp Wiesner, and Odej Kao. Logrca: Log-based root cause analysis for distributed services. In *European Conference on Parallel Processing*, pages 362–376. Springer, 2024.

[11] Ziming Zhao, Zhenwei Wang, Tiehua Zhang, Zhishu Shen, Hai Dong, Zhen Lei, Xingjun Ma, Gaowei Xu, Zhijun Ding, and Yun Yang. Chase: A causal hypergraph based framework for root cause analysis in multimodal microservice systems. *arXiv preprint arXiv:2406.19711*, 2024.

[12] Mingjie Li, Zeyan Li, Kanglin Yin, Xiaohui Nie, Wenchi Zhang, Kaixin Sui, and Dan Pei. Causal inference-based root cause analysis for online service systems with intervention recognition. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 3230–3240, 2022.

[13] Min Du, Feifei Li, Guineng Zheng, and Vivek Srikumar. Deeplog: Anomaly detection and diagnosis from system logs through deep learning. In *Proceedings of the 2017 ACM SIGSAC conference on computer and communications security*, pages 1285–1298, 2017.

[14] Haixuan Guo, Shuhan Yuan, and Xintao Wu. Logbert: Log anomaly detection via bert. In *2021 international joint conference on neural networks (IJCNN)*, pages 1–8. IEEE, 2021.

[15] Lingzhi Wang, Nengwen Zhao, Junjie Chen, Pinnong Li, Wenchi Zhang, and Kaixin Sui. Root-cause metric location for microservice systems via log anomaly detection. In *2020 IEEE international conference on web services (ICWS)*, pages 142–150. IEEE, 2020.

[16] Paolo Notaro, Soroush Haeri, Jorge Cardoso, and Michael Gerndt. Logrule: Efficient structured log mining for root cause analysis. *IEEE Transactions on Network and Service Management*, 20(4):4231–4243, 2023.

[17] Siyang Lu, BingBing Rao, Xiang Wei, Byungchul Tak, Long Wang, and Liqiang Wang. Log-based abnormal task detection and root cause analysis for spark. In *2017 IEEE International Conference on Web Services (ICWS)*, pages 389–396. IEEE, 2017.

[18] Thorsten Wittkopp, Alexander Acker, and Odej Kao. Progressing from anomaly detection to automated log labeling and pioneering root cause analysis. In *2023 IEEE International Conference on Data Mining Workshops (ICDMW)*, pages 1231–1239. IEEE, 2023.

[19] Max Landauer, Sebastian Onder, Florian Skopik, and Markus Wurzenberger. Deep learning for anomaly detection in log data: A survey. *Machine Learning with Applications*, 12:100470, 2023.

[20] Ruyue Xin, Peng Chen, and Zhiming Zhao. Causalrca: Causal inference based precise fine-grained root cause localization for microservice applications. *Journal of Systems and Software*, 203:111724, 2023.

[21] Yongqian Sun, Binpeng Shi, Mingyu Mao, Minghua Ma, Sibo Xia, Shenglin Zhang, and Dan Pei. Art: A unified unsupervised framework for incident management in microservice systems. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, pages 1183–1194, 2024.

[22] Li Wu, Johan Tordsson, Erik Elmroth, and Odej Kao. Microrca: Root cause localization of performance issues in microservices. In *IEEE/IFIP Network Operations and Management Symposium (NOMS)*, 2020.

[23] Cisco Systems, Inc. *RADIUS Interim Update at Call Connect*. Cisco, 2016. Cisco IOS XE Security Configuration Guide.

[24] Cisco Systems, Inc. *Cisco Secure Firewall ASA NetFlow Implementation Guide*. Cisco, 2022. NetFlow Secure Event Logging (NSEL) Documentation.

[25] Elastic. *Potential Successful SSH Brute Force Attack*. Elastic Security, 2024. Elastic Security Rule Documentation.

[26] Tao Ban, Takeshi Takahashi, Samuel Ndichu, and Daisuke Inoue. Breaking alert fatigue: Ai-assisted siem framework for effective incident response. *Applied Sciences*, 13(11):6610, 2023.

A Link to source code

We uploaded the code with all the implementation (including data extraction automation and sampling) on Github: `https://github.com/patriciaalonso100/Master-Thesis-DTU---s232519-.git`